Performance Analysis of Sparse Matrix-Vector Multiplication (SpMV)

Michael Bernasconi
ID Student: 267681
michael.bernasconi@studenti.unitn.it
https://github.com/Michael-Bernasconi/Sparse-Matrix-Vector-Multiplication
Department of Information Engineering and Computer Science
University of Trento

***Abstract*——-**
***Index Terms*—component, formatting, style, styling, insert**

I. Introduction

Problem statement, why SpMV matters, and the question your investigation addresses.

- **Introduction:** What SpMV is, why performance is memory-bound, and the scope of the study.

II. Methodology

Formats tested, CPU/GPU implementations, validation method, measurement methodology, and hardware/software environment.

- **Methodology:**
  - Describe the formats (CSR-cpu, COO-gpu, CSR-gpu, CSR-vector-gpu, cuSPARSE).
  - Explain your kernels: Base COO, Base CSR, and your **Optimized CSR (explaining how you use shared memory and warps)**.
  - Describe the validation method (tolerance on `float32`).

III. Dataset

Short characterization of the 10 SuiteSparse matrices and the random dense vector input.

- **Dataset:**
  - Insert a **Summary Table**. Recommended columns: `Matrix Name | M (Rows) | N (Cols) | NNZ | Average NNZ per row | Row length variance | Description`.
- State that the vector is random(42) with a fixed seed for reproducibility.
- **Rigorous validation of results:**
  - Write a function `validate_results(float *y_cpu, float *y_gpu, int M)`. Since you are using floating-point numbers (`float`), use a **tolerance** (e.g., $epsilon = 1e-3$). Implement a `for` loop over the entire vector: if $abs(y\_cpu[i] - y\_gpu[i]) > epsilon$, print "FAIL", otherwise "PASS".
  - Explain this criterion in the report: result validation depends on the matrix type and the data representation method.
- Hardware architecture.
- **Theoretical Explanation in Validation:** In parallel architectures (GPUs), the order in which threads execute additions (e.g., via `atomicAdd` or reductions) is non-deterministic. Because floating-point arithmetic is non-associative, different summation orders generate slightly different rounding errors.
- **Comparison with Peak Bandwidth (Peak Memory Throughput):** In the Bandwidth (BW) plots, add a horizontal line or perform a comparison with the Theoretical Peak Bandwidth of the GPU (e.g., A30 is $\sim$933.1 GB/s).

IV. Experimental Setup

- Experiments.
- Hardware and software / library used / dataset.

V. Results

Plots/tables for runtime and FLOPS/GFLOP/s; optional memory/cache indicators.

- **Results:**
  - Create bar charts (e.g., GFLOP/s for each matrix: Base CSR, Base COO, Opt CSR, cuSPARSE) and include the calculation formulas.
  - Check and show how to present average time and standard deviation, explaining the methodology as done in the documentation.
  - Create BW plots, time-to-solution plots, and charts or tables for cache hit/miss (with relative formulas).
  - **INCLUDE FORMULAS FOR EVERY METRIC**.
  - Include data on the measured temporal variability.

To mitigate the variability of the shared environment, each benchmark was submitted to the cluster 5 independent times, reporting the results of the best execution (best-of-5 minimum execution time). To ensure numerical consistency between the CSR (write-only) and COO (atomic-accumulate) formats, an output vector reset was included within the benchmark loop.

VI. Discussion

Interpret the results in terms of format properties, matrix structure, and memory behavior.

- **Discussion:** Explain the *why* behind the results.
- *Load Balancing:* Explain how CSR-Vector outperforms CSR-Base in irregular matrices.
- *Memory Behavior:* Cite `ncu` data to demonstrate that optimizations reduce cache misses.
- Explain why cuSPARSE is even faster (adaptive techniques).

VII. Conclusion

Main lessons learned, limitations, and practical takeaways.

- **Conclusion:** Main takeaways. Explain under which matrix characteristics and memory access conditions one kernel is preferable over another.

References

References

[1] Gao, J., Liu, B., Ji, W. and Huang, H., 2024. A systematic literature survey of sparse matrix-vector multiplication. arXiv preprint arXiv:2404.06047.

[2] Chu, G., He, Y., Dong, L., Ding, Z., Chen, D., Bai, H., Wang, X., and Hu, C. Efficient Algorithm Design of Optimizing SpMV on GPU. In HPDC '23, 2023.

[3] Bell, N., and Garland, M. Implementing Sparse Matrix-Vector Multiplication on Throughput-Oriented Processors. In SC '09, 2009.

[4] Greathouse, J. L., and Daga, M. Efficient Sparse Matrix-Vector Multiplication on GPUs Using the CSR Storage Format. In SC14, 2014.

[5] Liu, W., and Vinter, B. CSR5: An Efficient Storage Format for Cross-Platform Sparse Matrix-Vector Multiplication. In ICS '15, 2015.

[6] Merrill, D., and Garland, M. Merge-Based Parallel Sparse Matrix-Vector Multiplication. In SC16, 2016.

[7] M. Young, The Technical Writer's Handbook. Mill Valley, CA: University Science, 1989.